

Academic year 2025-2026 (ODD Sem)

Date: 06/11/2025	CIE - 1	Max. Marks: 10+50=60M
Semester: VII	UG	Duration: 2 Hours
Course: Computer Graphics & Virtual Reality		Code: CS373IA

SCHEME & SOLUTIONS

Q. No.	QUIZ	M
1.1	Write the difference between Computer Graphics and Image Processing. ----- (0.5M X2 = 1M) - The main difference between image processing and computer graphics is that image processing is concerned with the manipulation of digital images, while computer graphics are concerned with the creation of images. - Image processing algorithms are used in order to improve the quality of digital images, or to extract specific information from them. For example, image processing can be used to remove noise from an image, or to sharpen it. Computer graphics, on the other hand, are used to generate images from scratch, or to modify existing images.	1
1.2	Write a neat diagram representing the five classic components of a VR system. VR System Architecture The five classic components of a VR system	1
1.3	List the major groups of graphics functions and give an example OpenGL API/function. The major groups of graphic functions: 1. Primitive functions 2. Attribute functions 3. Viewing functions 4. Transformation functions 5. Input functions 6. Control functions 7. Query functions	2
1.4	Write the generic main program in C (using OpenGL) that works for most noninteractive graphics applications. <pre>#include <GL/glut.h> void main(int argc, char **argv) { glutInit(&argc,argv); glutInitDisplayMode (GLUT_SINGLE GLUT_RGB); glutInitWindowSize(500,500); glutInitWindowPosition(0,0); glutCreateWindow("simple OpenGL example");</pre>	2



	<pre>glutDisplayFunc(display); myinit(); glutMainLoop(); }</pre>
1.5	Write neat sketches to represent the resultant output of successive generations of 2-D Sierpinski Gasket number of recursive steps $k = 0, 1, 2, 3$ respectively. ----- (0.25M X4 =2M) S_0 S_1 S_2 S_3 Successive generations of the Sierpinski gasket.
1.6	Write neat sketches to represent color formation in additive color model and subtractive color model. ----- (1 M X2 =2M) (a) Additive color. (b) Subtractive color.

Q. No.	TEST	M
1a.	Diagram of the graphics pipeline ----- 2 M Explanation of the four major steps in the imaging process: ----- 3 M i) Vertex processing ii) Clipping and primitive assembly iii) Rasterization iv) Fragment processing Vertices Vertex processor Clipper and primitive assembler Rasterizer Fragment processor Pixels	05
1b.	i) Size of 8 inch by 10 inch screen with a resolution of 100 pixels per inch in each direction. If we want to store 6 bits per pixel in the frame buffer Size of frame buffer = Resolution X bits per pixel = 100 pixels/inch in each direction X 6 bits/pixels ii) Aspect ratio of 1024x768 image is 768/1024 Height of image with 640 pixels and same aspect ratio 768/1024 is Aspect ratio = width / height $\Rightarrow$ Height = 640 X 768/1024 = 480 pixels	(2+2+1=5)

ii) The RGB coordinates of a color at (0.15, 0.75, 0) in the CMY space is $(1 - 0.15, 1 - 0.75, 1 - 0)$
 $= (0.85, 0.25, 1)$

- 2a. Determine the vertex sequence to draw the Figure 2a using varying OpenGL Line drawing primitives. Write the OpenGL code segment for all the variations. ----- 4M

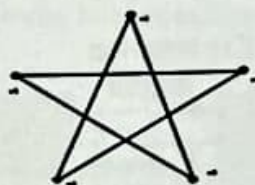


Figure 2a

Find the minimum number of glVertex() calls necessary to draw the figure comparing all the variations of OpenGL Line drawing primitives ----- 1M

Primitive	Description	Example Connection	Typical Use	No. of Vertices	glVertex () Calls
GL_LINES	Each pair of vertices forms an independent line	(v1,v2), (v3,v4)...	Disjoint lines	5 segments	10
GL_LINE_STRIP	Connects each vertex to the next	v1-v2-v3-v4...	Connected open path	6	6
GL_LINE_LOOP	Like GL_LINE_STRIP, but automatically closes the shape	v1-v2-v3-v4-v1	Closed polygon outline	5	5

To draw Figure 2a (a closed connected shape), the minimum number of glVertex() calls is 5, when using GL_LINE_LOOP

- 2b. Definition of Virtual Reality ----- 1M
 Discuss the Three I's of Virtual Reality ----- 4M
 Virtual reality is a high-end user-computer interface that involves real-time simulation and interactions through multiple sensorial channels. These sensorial modalities are visual, auditory, tactile, smell, and taste.
 Virtual reality is an integrated trio of immersion interaction-imagination --- explain

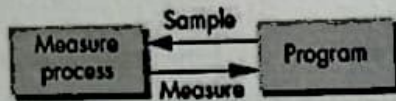


The three I's of virtual reality, immersion-interaction-imagination.

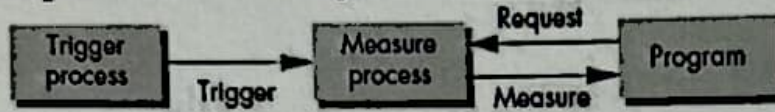
3a.

Sample Mode — Figure + Explanation

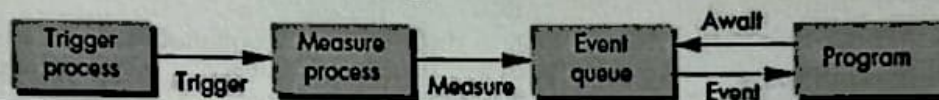
— 1.5 M

**Request Mode — Figure + Explanation**

— 1.5 M

**Event Mode — Figure + Explanation**

— 2 M



3b.

All 3D trackers, regardless of the technology they use, share a number of very important performance parameters. Discuss the same illustrating with neat sketches.

05

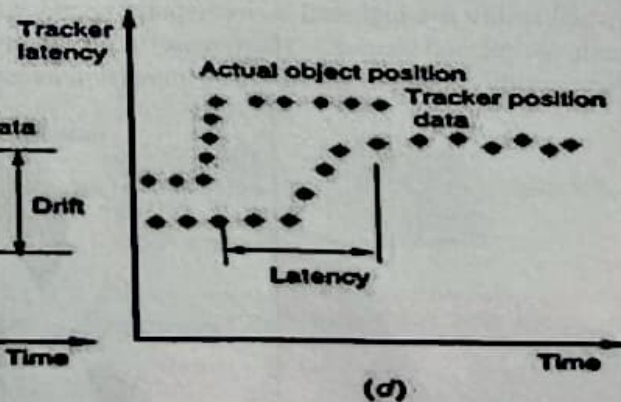
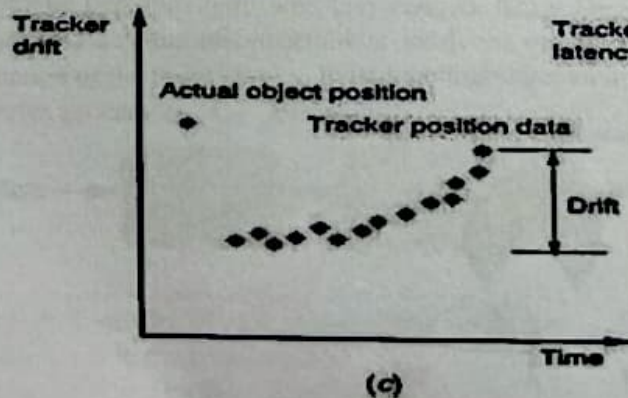
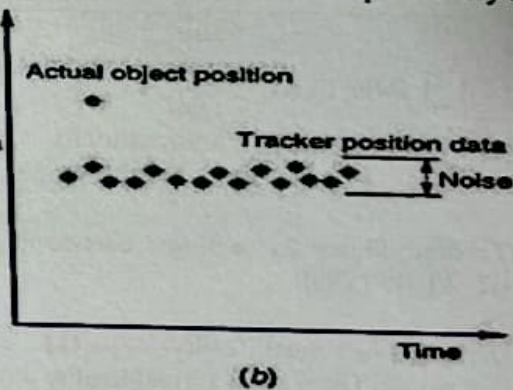
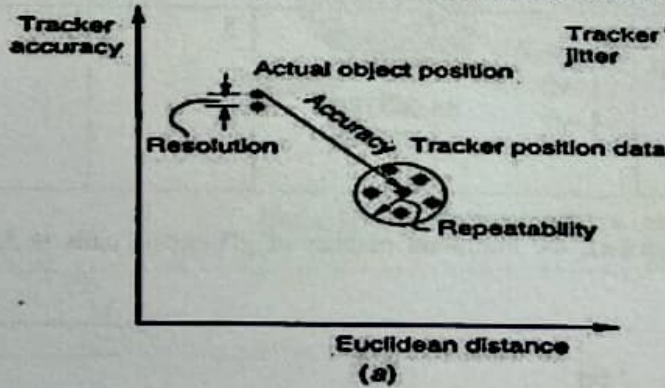
Tracker Performance Parameters

Tracker accuracy represents the difference between the object's actual 3D position and that reported by tracker measurements.

Tracker jitter represents the change in tracker output when the tracked object is stationary.

Tracker drift is the steady increase in tracker error with time.

The tracker update rate represents the number of measurements (datasets) that the tracker reports every second.



Tracker performance parameters: (a) accuracy; (b) jitter; (c) drift; (d) latency.

4.

Construct an approximation to a 3D sphere with appropriate mathematical formulations. ----- 2M

10

Write the complete C program to approximate a sphere using OpenGL primitives. ----- 7M
Write the expected output of the same pictorially. ----- 1M

Latency is the time delay between action and result. In the case of the 3D tracker, latency is the time between the change in object position/orientation and the time the sensor detects this change.

Construct an approximation to a sphere is to use a set of polygons defined by lines of longitude and latitude. We can do so very efficiently using either quad strips or triangle strips.

Consider a unit sphere described by the following three equations: ----- 2M

$$x(\theta, \phi) = \sin \theta \cos \phi,$$

$$y(\theta, \phi) = \cos \theta \cos \phi,$$

$$z(\theta, \phi) = \sin \phi.$$

If we fix θ and draw curves as we change ϕ , we get circles of constant longitude. ----- 3M

Likewise, if we fix ϕ and vary θ , we obtain circles of constant latitude.

By generating points at fixed increments of θ and ϕ , we can define quadrilaterals, the code for the quadrilaterals corresponding to increments of 20 degrees in θ and to 20 degrees in ϕ is

```
for(phi=-80.0; phi<=80.0; phi+=20.0)
{
    phir=c*phi;
    phir20=c*(phi+20);
    glBegin(GL_QUAD_STRIP);
    for(theta=-180.0; theta<=180.0; theta+=20.0)
    {
        thetar=c*theta;
        x=sin(thetar)*cos(phir);
        y=cos(thetar)*cos(phir);
        z=sin(phir);
        glVertex3d(x,y,z);
        x=sin(thetar)*cos(phir20);
        y=cos(thetar)*cos(phir20);
        z=sin(phir20);
        glVertex3d(x,y,z);
    }
    glEnd();
}
```

At the poles, because all lines of longitude converge there, use two triangle fans, one at each pole as follows: ----- 2.5 M

```
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0, 0.0, 1.0);
c=M_PI/180.0;
c80=c*80.0;

z=sin(c80);
for(theta=-180.0; theta<=180.0; theta+=20.0)
{
    thetar=c*theta;
    x=sin(thetar)*cos(c80);
    y=cos(thetar)*cos(c80);
    glVertex3d(x,y,z);
}
glEnd();
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0, 0.0, -1.0);
z=-sin(c80);
for(theta=-180.0; theta<=180.0; theta+=20.0)
{
```

(2+
7+1
)

```
thetar=c*theta;
x=sin(thetar)*cos(c80);
y=cos(thetar)*cos(c80);
glVertex3d(x,y,z);
}
glEnd();
```

Suitable main () program

----- 1.5M

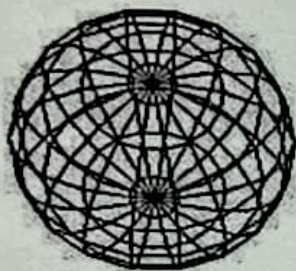


Figure : Sphere approximation with quadrilaterals.

----- 1M

- 5.** OpenGL program to create the hierarchical menu and attach the corresponding operations to each menu with mouse left button ----- 3M
- Write appropriate code for each menu entries:
- Two-D Sierpinski Gasket (Random Approach) ----- 3 M
- Two-D Sierpinski Gasket (Recursive Approach) ----- 4M